JAI BYOK and subscriptions: deployment setup

This implementation stores OpenAI, OpenRouter, and Google Gemini API keys in the signed-in user's browser `localStorage`, namespaced by Supabase user ID. Keys are not written to Supabase or another JAI database. When a JAI server-side feature needs a provider request, the browser sends the key in a per-request HTTPS header. JAI uses it in memory and does not intentionally log or persist it. The selected provider receives the AI request and bills model usage separately from any JAI subscription.

Important browser-security limitation: `localStorage` is not encrypted and is readable by scripts running on the JAI origin, browser extensions, and anyone with access to that browser profile. Do not use this option on a shared or untrusted device. For higher protection, consider a user-controlled OS keyring in a native desktop client; browser-side encryption does not protect against same-origin script compromise if the decryption secret is also available to the page.

Do not enable paid enforcement until setup is complete. The Render configuration keeps the subscription and BYOK gates off by default.

1. Apply the Supabase migration

Apply `supabase/migrations/20261101000000_add_byok_and_subscriptions.sql` to the Supabase project used for JAI Auth. It creates only the `jai_subscriptions` table and a signed-webhook update function. It does **not** create a provider-key table or store API keys. Row-level security is enabled; the server uses the Supabase service-role key only for subscription state.

2. Configure server secrets

In the hosting provider's secret store (Render for the deployment described in `render.yaml`), configure:

- Existing `SUPABASE_URL`, `SUPABASE_ANON_KEY`, and `SUPABASE_JWT_SECRET` for authentication.
- `SUPABASE_SERVICE_ROLE_KEY` for subscription state only. Keep it server-side; never expose it to the browser.
- `STRIPE_SECRET_KEY`, initially using Stripe test mode.
- `STRIPE_WEBHOOK_SECRET` for the JAI webhook endpoint.
- `STRIPE_PRICE_ID` for the recurring JAI price. Configure the amount and billing interval in Stripe, not in this repository.

- `JAI_PUBLIC_URL=https://j-ai.top`.

There is no `JAI_BYOK_ENCRYPTION_KEY`: JAI does not store user provider keys.

3. Configure Stripe

Create a recurring JAI product/price in Stripe and configure the Stripe Customer Portal. Add a webhook endpoint at:

`https://j-ai.top/api/billing/webhook`

Subscribe it to `customer.subscription.created`, `customer.subscription.updated`, and `customer.subscription.deleted`. Save its `whsec_...` secret as `STRIPE_WEBHOOK_SECRET`. The Stripe webhook is the source of truth for entitlements; only `active` and `trialing` subscriptions are accepted. No payment is initiated by applying this code or migration.

4. Test BYOK and subscription behavior

In a non-production deployment, sign in and open **AI Settings**. Select OpenAI, OpenRouter, or Google Gemini, enter a model ID and key, and choose **Test and save on this browser**. JAI sends a one-shot validation request to the provider's models endpoint; it does not write the key to the database. After successful validation, the browser stores the key and model in localStorage for that signed-in account. Chat, image, and voice requests attach the currently selected provider, model, and key in custom HTTPS headers. Some built-in tool commands are handled locally by JAI and do not make a model call, even though the browser sent the request headers to JAI.

If the key is removed in AI Settings, it is removed from that browser's localStorage. For incident response, also revoke the key at the AI provider. Signing out does not erase local keys, but another signed-in account on the same browser uses a separate localStorage namespace.

The JAI server does receive the key transiently for requests handled by its server-side tools; it is not a direct browser-to-provider connection. The API key is not placed in URLs or request bodies. Application code does not log the header or provider exception details. Confirm that reverse proxies, observability tools, and hosting configuration do not capture request headers before production launch.

5. Enable enforcement only after setup

After applying the subscription migration, configuring Supabase and Stripe, deploying, and testing, set:

```
Plain Text
```

```
JAI_REQUIRE_ACTIVE_SUBSCRIPTION=true
JAI_REQUIRE_BYOK=true
```

With subscription enforcement enabled, protected API features require a verified signed-in user with an active subscription; authentication, billing, health, settings, and key-test flows remain available as configured. With BYOK enforcement enabled, chat, image analysis, and voice responses require a valid provider configuration in the request. Leave either flag off during staging until its prerequisites are ready.

Operational notes

- Keys and model selections are device/browser-specific and are not synchronized between devices.
- OpenAI-compatible chat/image requests use the selected provider's fixed HTTPS endpoint. Gemini chat and image analysis use Google's fixed `generateContent` endpoint with the key in `x-goog-api-key`.
- The initial Gemini model is `gemini-3.8-flash`; check Google's official Gemini model guide for current model IDs and lifecycle changes.
- Unsupported model modalities fail with a sanitized message; BYOK requests do not silently fall back to JAI's shared model key.
- Voice transcription uses the user's OpenAI key for OpenAI Whisper when OpenAI is selected. With other providers, the existing speech-recognition fallback is used before the resulting text is sent to the selected model.
- Privacy and terms pages disclose browser-local storage and the transient JAI request path. They should receive legal review before a paid launch.
- The code change does not update Render's live environment, apply the Supabase migration, configure Stripe, push to GitHub, or activate paid access.